

SMS

Student Management System

Project Report

Python Development — Task Submission

Technology Stack	Python 3, Streamlit, JSON
Interfaces	Command-Line (CLI) + Web GUI
Core Concepts	OOP, File Handling, Exception Handling
Repository Type	Public GitHub Repository

1. Objective

The objective of this project is to develop a Python-based application that efficiently manages student records. The system allows users to add, update, delete, search, and filter student information through both a command-line interface and a web-based graphical interface, while applying core software engineering principles: Object-Oriented Programming, modular code structure, file-based persistence, and robust exception handling.

2. Key Features

- **Student CRUD Operations** — Create, view, update, and delete student records.
- **Object-Oriented Programming** — A dedicated `Student` class encapsulates data and validation logic.
- **File Handling** — Records are persisted to a JSON file (`data/students.json`) and reloaded automatically on startup.
- **Search & Filter** — Search by name (partial match), filter by course or grade, and sort by marks.
- **Exception Handling** — Custom exceptions (`StudentNotFoundError`, `DuplicateStudentError`, `InvalidInputError`) prevent crashes and give clear feedback.
- **Modular Code Structure** — Logic is separated into model, persistence, business-logic, and interface layers.
- **Dual Interface** — The same backend logic powers both a CLI (`main.py`) and a Streamlit web GUI (`app.py`).

3. System Architecture

The project follows a layered, modular architecture so that each file has a single responsibility. This makes the code easier to test, extend, and reuse across both interfaces.

File	Responsibility
<code>student.py</code>	Student model class — attributes, field validation, grade calculation
<code>exceptions.py</code>	Custom exception classes for domain-specific errors
<code>file_handler.py</code>	Reads/writes student data to a JSON file; handles missing/corrupt files
<code>student_manager.py</code>	Business logic — CRUD, search, filter, sort operations
<code>main.py</code>	Command-line interface (menu-driven)
<code>app.py</code>	Web GUI interface (Streamlit) — reuses the same manager/model layer

Data Flow

Both interfaces (CLI and Web GUI) call the same `StudentManager` class, which validates operations, updates the in-memory list of `Student` objects, and persists changes to `data/students.json` via `FileHandler` after every change. This avoids duplicated logic and keeps both interfaces perfectly in sync.

Grading Logic

Grades are computed automatically from marks: A+ (90 and above), A (75-89), B (60-74), C (40-59), F (below 40).

4. Exception Handling

Three custom exception classes handle domain-specific error conditions so that the application never crashes on invalid input or missing records:

Exception	Raised When
InvalidInputError	A field fails validation (empty name, non-numeric age/marks, out-of-range values)
DuplicateStudentError	An Add operation uses a Student ID that already exists
StudentNotFoundError	A lookup, update, delete, or search finds no matching record

In both the CLI and GUI, these exceptions are caught at the interface layer and shown as a clear, user-facing message instead of a raw traceback.

5. Working Screens

5.1 Command-Line Interface

The CLI provides a numbered menu covering all core operations. Below is an actual run showing the menu and the "View All Students" output.

```
python main.py

$ python main.py
=====
      STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View All Students
3. Update Student
4. Delete Student
5. Search Student by Name
6. Filter by Course
7. Filter by Grade
8. Sort by Marks
9. Exit
=====

Enter your choice (1-9): 2

--- All Students ---
-----
ID: S001 | Name: Alice Sharma | Age: 20 | Course: CS | Marks: 92.0 | Grade: A+
ID: S002 | Name: Bob Kumar | Age: 22 | Course: ECE | Marks: 67.0 | Grade: B
ID: S003 | Name: Charlie Verma | Age: 21 | Course: CS | Marks: 45.0 | Grade: C
ID: S004 | Name: Divya Patel | Age: 19 | Course: ME | Marks: 78.0 | Grade: A
ID: S005 | Name: Esha Reddy | Age: 23 | Course: CS | Marks: 88.0 | Grade: A
-----
Total: 5 student(s)

Press Enter to continue...
```

Figure 1 — CLI menu and student list output

Deploy ⋮

Student Management System — built with Python, OOP, and Streamlit

Deploy ⋮

Deploy ⋮

Student Management System — built with Python, OOP, and Streamlit

Figure 3 — All student records displayed in a live data table

5.4 Web GUI — Update Student

Deploy 

 **Student Management System**

Python OOP • File Handling • Exception Handling • Search & Filter

 Add  View All  Update  Delete  Search  Filter  Sort

Update Student

Select Student ID

S001 

Name

Alice Sharma

Course

CS

Age

20

Marks

92.0

Update Student

Student Management System — built with Python, OOP, and Streamlit

Figure 4 — Update form pre-filled with the selected student's current data

5.5 Web GUI — Delete Student

Deploy 

 **Student Management System**

Python OOP • File Handling • Exception Handling • Search & Filter

 Add  View All  Update  Delete  Search  Filter  Sort

Delete Student

Select Student ID to delete

S001 

Delete Student

Student Management System — built with Python, OOP, and Streamlit

Figure 5 — Delete Student screen

5.6 Web GUI — Search

Deploy 

Student Management System

Python OOP • File Handling • Exception Handling • Search & Filter

 Add  View All  Update  Delete  Search  Filter  Sort

Search by Name

Enter name or part of it

Student Management System — built with Python, OOP, and Streamlit

Figure 6 — Search by name

5.7 Web GUI — Filter

Deploy 

Student Management System

Python OOP • File Handling • Exception Handling • Search & Filter

 Add  View All  Update  Delete  Search  Filter  Sort

Filter Students

Filter by course (e.g. CS)

Filter by grade

Choose an option



Student Management System — built with Python, OOP, and Streamlit

Figure 7 — Filter by course and grade

5.8 Web GUI — Sort by Marks

Student Management System

Python OOP • File Handling • Exception Handling • Search & Filter

[+](#) Add [📄](#) View All [✏️](#) Update [🗑️](#) Delete [🔍](#) Search [🎯](#) Filter [🔼](#) **Sort**

Sort by Marks

Order

☒ High to Low ☐ Low to High

ID	Name	Age	Course	Marks	Grade
S001	Alice Sharma	20	CS	92	A+
S005	Esha Reddy	23	CS	88	A
S004	Divya Patel	19	ME	78	A
S002	Bob Kumar	22	ECE	67	B
S003	Charlie Verma	21	CS	45	C

Student Management System — built with Python, OOP, and Streamlit

Figure 8 — Students sorted by marks (high to low)

6. Deployment

The web GUI (app.py) is deployed on Streamlit Community Cloud, a free hosting platform for Streamlit applications that deploys directly from a GitHub repository.

Live application link:

[\[Add your deployed Streamlit Cloud URL here\]](#)

GitHub repository link:

[\[Add your public GitHub repository URL here\]](#)

7. Challenges & Solutions

Keeping CLI and GUI in sync

Solved by putting all business logic in `student_manager.py` and having both `main.py` and `app.py` call the same methods, rather than duplicating logic.

Preventing crashes on bad input

Solved with field-level validation inside the `Student` class and custom exceptions caught at the interface layer.

Handling corrupted or missing data files

`FileHandler` catches `FileNotFoundError` and `JSONDecodeError` and falls back to an empty record set instead of crashing.

8. Conclusion

The Student Management System successfully meets all specified requirements: full CRUD functionality, Object-Oriented design, persistent file-based storage, search and filter capabilities, and comprehensive exception handling — all within a clean, modular code structure. The addition of a Streamlit-based web GUI alongside the original CLI demonstrates that the same core logic can reliably power multiple interfaces, and the project is deployed live for easy access and demonstration.

— End of Report —